Alex Scarlatos
5/9/2016

Image Quilting for Texture Synthesis and Transfer

Abstract

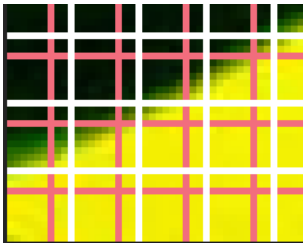
For this project, I implemented the image quilting method of generating textures. It essentially cuts up an image into blocks, and pastes these blocks in a way that produces a seemingly fluid texture of any size. This method is then extended to perform texture transfer, which is placing these blocks in such a way that they appear to mimic another image (re-drawing an image with a texture). There are a few practical applications for this. Texture transfer allows you to create unique effects for any image, and texture synthesis could be used for texture mapping models. However, while experimenting with the process, I found that the texture synthesis method can also produce completely unique images with their own structures and properties.

Method

There are two basic steps for this algorithm. They are run in tandem, but it is easiest to explain them as one after the other. First, the algorithm cuts up the source image into a grid, where the grid cells overlap. The size of each block and the size of each overlap are parameters entered by the user. The algorithm will first pick a random block from the source image to paste into the new texture. From there, it will find the next block to place (blocks are placed row by row, starting in the bottom left corner). The best block to place is the one with the least amount of error in the overlapping region. To find the error of a border, sum all the pixel errors in that border.

$PixelError_{r,c} = sqrt((Red(LeftBorder_{r,c}) - Red(RightBorder_{r,c}))^2 + GreenDif^2 + BlueDif^2)$

The algorithm analyzes the errors of overlapping borders between every block and the blocks that are surrounding the spot to be placed. It then chooses a random block with a low overlap error. Then the process continues until the area is filled.



The white lines represent where a normal grid would be, and the pink lines represent where actual blocks are found. In the first column and row the block starts at the edge of the image.
The indexing formula is blockPos = [column * (blockSize – borderSize), row * (blockSize – borderSize)]

After we have the block to place, we have to define a custom border between the blocks. The border is made to have the least amount of error so that you can't tell that the blocks are merely pasted next to each other and the texture can appear fluid. Using dynamic programming this path can be efficiently found. The formula is $PathError_{r,c} = PixelError_{r,c} + min(PathError_{r+1,c-1},$

$PathError_{r+1,c}$, $PathError_{r+1,c+1}$) where $PathError_{r,c} = PixelError_{r,c}$ for the top row, and we construct the matrix by getting the path error at every pixel in row 0. While the matrix is being constructed, we fill in a path matrix for each pixel that says which following column will yield the minimum error. The path then can be easily reconstructed by starting at the lowest path error pixel in the first row.

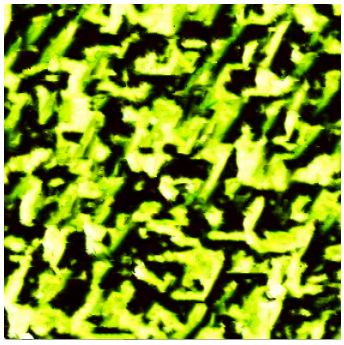
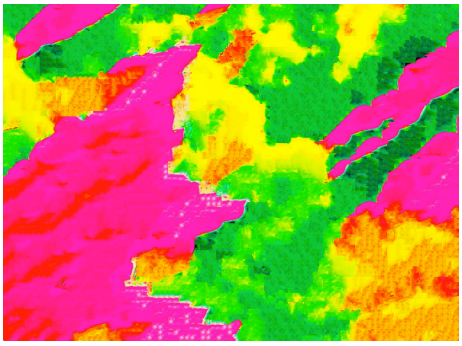
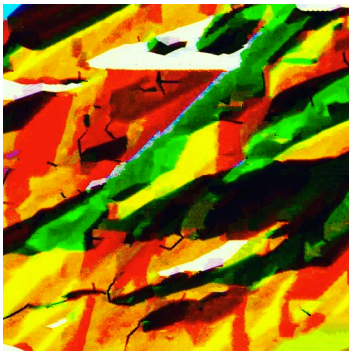
The process is slightly different for texture transfer. When choosing a block, you must also consider the luminescence error between the block being chosen and the block at the spot being placed in the target image.

$PixelError_{r,c}$ += Luminescence($TargetImage_{y+r,x+c}$) - Luminescence($Block_{r,c}$), where x and y are where the block is being placed.

However, for choosing a border path, I found that it was best to consider the minimal error of the area between the left and right border areas and the corresponding area in the target image. The formula for constructing the error matrix is different in this case.
$PixelError_{r,c} = sqrt((Red(LeftBorder_{r,c}) - Red(RightBorder_{r,c}))^2 + GreenDif^2 + BlueDif^2)$
$AreaPixelError_{r,c} = Sum_{\text{for each } i<c}(PixelError_{r,i}) + Sum_{\text{for each } i>=c}(PixelError_{r,i})$
This will give you the area error of the row at each pixel, so after performing dynamic programming on this matrix you will get the border with the minimum area between the corresponding area on the target image.

Synthesis Results



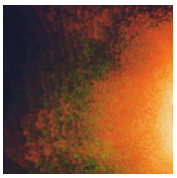
Transfer Results

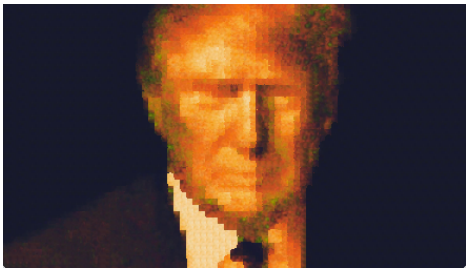


Downloaded from http://i2.cdn.turner.com/cnnnext/dam/assets/160118134132-donald-trump-nigel-parry-large-169.jpg



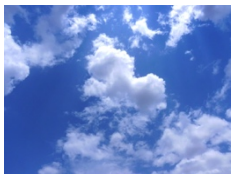








Downloaded from http://az616578.vo.msecnd.net/files/2016/04/23/635969831640460799-54382534_sanders%20again.png



Downloaded from http://rozup.ir/up/banerbazar/tekscher/clouds_texture01.jpg







Reference:
Alexei A., Efros, and Freeman William T. "Image Quilting For Texture Synthesis And Transfer." (2010): *BASE*.